

CookPilot 상위 수준 전체 흐름 지도

주석 달린 코드 → 기능별 텍스트 흐름도(13개) → **이 문서** → Mermaid Flowchart → HIPO 기능명세서. 이 문서는 기능 하나를 **단일 단위**로 보고 구조 · 데이터 · 화면 이동만 표시한다. 세부 실행 단계는 반복하지 않는다 — docs/flow/flow.md 의 같은 FN 항목을 참조한다.

참조 규칙 (기존 문서와 동일)

파일:F#	단계 참조. F 번호는 파일마다 1부터 다시 매긴다	:: 함수() :F#	함수 참조. 전체 경로를 함께 적는다	→ · ↓	데이터 · 실행 흐름
[true] [false]	조건 분기	[반복]	반복	▷	외부 시스템 · API · 저장소 접근

이 문서에서 새로 부여하는 ID (이후 단계에서 그대로 재사용)

L1 – L6	계층 — 도메인 · 유스케이스 · 어댑터 · 서버액션 · 라우트 · 화면	D1 – D4	데이터 — localStorage · 테이블 · 버킷 · 외부 API	J1 – J7	화면 여정
S1 – S5	시스템 — Browser · Next Server · Supabase · Gemini · Pinecone	FN01 – FN13	기능. ID 와 순서는 docs/flow/flow.md 와 동일	확인 필요	코드에서 근거를 찾지 못한 위치

기능 13종

ID	기능 이름 (13개 기능별 흐름도 docs/flow/flow.md 의 장 이름과 같다)	ID	기능 이름
FN01	Gemini API Key 입력	FN08	댓글
FN02	로그인 · 회원가입	FN09	좋아요 · 즐겨찾기
FN03	음성 입력 레시피 생성	FN10	프로필 관리
FN04	YouTube 링크 레시피 생성	FN11	AI 요리 상담 (RAG)
FN05	냉장고 재료 레시피 생성	FN12	저장 레시피 관리
FN06	음성 기반 요리 진행	FN13	커뮤니티 목록 · 검색 · 태그
FN07	게시글 작성 · 수정 · 삭제		

도형 읽는 법 — 둥근 상자=단말 · 네모=처리 · 겹줄 네모=다른 계층 호출 · 평행사변형=입력 · 출력 · 마름모=분기 · 원=연결점 · 원통=저장소 · 점선 화살표=서버를 거치지 않는 경로.

기능별 계층 · 파일 대응 — FN01 ~ FN07

칸에는 **파일 이름만** 적는다. 앞에 붙는 경로와 확장자는 왼쪽 계층 칸에 표기했다. **공유** 는 다른 기능도 함께 사용하는 파일이라는 뜻이며, 어느 기능이 공유하는지는 다음 쪽에 정리했다. · 는 해당 계층을 사용하지 않는다는 뜻이다.

계층	FN01 Gemini API Key 입력	FN02 로그인 · 회원가입	FN03 음성 입력 레시피 생성	FN04 YouTube 링크 레시피 생성	FN05 냉장고 재료 레시피 생성	FN06 음성 기반 요리 진행	FN07 게시글 작성 · 수정 · 삭제
L1 도메인 lib/domain/ *.ts	api-key	credentials display-name	recipe공유 shopping	recipe공유	recipe공유	cook-progress voice-tone gemini-model recipe공유	post-draft recipe-to-post post공유
L2 유스케이스 lib/usecase/ *.ts	enter-with-api-key	sign-in sign-up	plan-recipe공유	plan-recipe공유	plan-recipe공유	cook-along choose-cook-setup enter-with-api-key공유	write-post
L3 어댑터 lib/adapter/ *.ts	browser-api-key-store	supabase-auth-gateway supabase-server-client 공유 pending-auth-gateway	browser-audio gemini-recipe-gateway공유 browser-recipe-draft-store공유	gemini-recipe-gateway공유 browser-recipe-draft-store공유	gemini-recipe-gateway공유 browser-recipe-draft-store공유	gemini-live-gateway browser-speech browser-audio공유 browser-cook-setup-store	supabase-post-gateway browser-cover-upload supabase-server-client 공유
L4 서버액션 app/actions/ *.ts	.	auth auth-state	.	.	.	.	post공유 post-state
L5 라우트 app/**/page.tsx · route.ts	/start	/login /signup	/pick /shop	/pick /shop	/pick /shop	/start/model /cook /cook/done	/write /posts/[id]/edit /cook/done
L6 화면 components/**/*.tsx	start/api-key-form	login/login-form login/login-chrome signup/signup-form auth/auth-field	pick/voice-console pick/pick-shell shop/shop-shell공유	pick/pick-cards공유 shop/youtube-source shop/shop-shell공유	pick/pick-cards공유	cook/cook-shell cook/live-console cook/cook-timer setup/setup-picker	write/write-form cook/done-shell공유 post/post-actions공유

읽어 둘 것 — FN03 · FN04 · FN05 는 겉모습만 다르고 L1~L3 구성이 거의 같다. 음성 · 영상 · 재료라는 **입력 방식만 다르고** lib/usecase/plan-recipe.ts 의 finish() 로 수렴한다. FN06 만 lib/adapter/gemini-live-gateway.ts(WebSocket)를 쓰고, 나머지 요리 기능은 gemini-recipe-gateway.ts(REST)를 사용한다.

FN08 ~ FN13

계층	FN08 댓글	FN09 좋아요 · 즐겨찾기	FN10 프로필 관리	FN11 AI 요리 상담 (RAG)	FN12 저장 레시피 관리	FN13 커뮤니티 목록 · 검색 · 태그
L1 도메인 lib/domain/ *.ts	post공유	.	avatar	ask review api-key공유	recipe-shelf recipe공유	community-tab post공유 avatar공유
L2 유스케이스 lib/usecase/ *.ts	discuss-post	react-to-post	rename-me	ask-kitchen	keep-recipe-shelf	enter-with-api-key공유 keep-recipe-shelf공유
L3 어댑터 lib/adapters/ *.ts	supabase-comment-gateway supabase-post-reader공유 browser-comment-store	supabase-reaction-gateway	browser-avatar-upload supabase-profile-gateway	langchain-rag csv-reviews supabase-chat-log browser-chat-id-store	browser-recipe-shelf-store browser-recipe-draft-store공유	supabase-post-reader공유 supabase-profile-gateway공유 supabase-reaction-gateway공유
L4 서버액션 app/actions/ *.ts	post공유	post공유	post공유	.	.	.
L5 라우트 app/**/page.tsx · route.ts	/posts/[id]	/posts/[id] /community	/account	/ask /api/chat /api/chat/[id] /api/kitchen/index	/shelf	/community /posts/[id]
L6 화면 components/**/*.*.tsx	post/live-talk post/post-talk	post/post-actions공유	write/photo-form write/account-form	ask/ask-shell	shelf/shelf-stand cook/done-shell공유	community/post-cards community/community-side community/home-me community/tag-link post/post-article post/live-post-view

FN13 에는 L2 유스케이스가 거의 없다. app/community/page.tsx 가 lib/adapters/supabase-post-reader.ts 를 직접 호출하고 도메인 함수로 필터링한다. 조회 전용이고 비즈니스 규칙이 없어 유스케이스 계층을 두지 않았다. 목록에 규칙이 추가되면 계층을 넣어야 한다. FN09 에는 L1 도메인이 없다. uuid 판정(looksLikePost)이 lib/usecase/react-to-post.ts:F1 에 있다.

재사용 지점 — 여러 기능이 공유하는 파일

계층 지도에서 공유 로 표시한 파일들이다. 이 파일이 변경되면 오른쪽 칸의 기능이 모두 영향을 받는다.

파일 (전체 경로)	이 파일을 사용하는 기능	공유 사유
lib/domain/recipe.ts	FN03 · FN04 · FN05 · FN06 · FN12	Recipe 구조를 아는 위치를 하나로 유지한다. 어느 입력 방식이든 같은 검증을 거친다.
lib/domain/post.ts	FN07 · FN08 · FN13	예시 게시글 16건 조회와 댓글 검증을 함께 담당한다.
lib/domain/api-key.ts	FN01 · FN11	브라우저 입력 시점과 /api/chat 수신 시점에 같은 규칙을 적용한다.
lib/domain/avatar.ts	FN10 · FN13	외부 도메인 URL 차단 규칙. 저장 시점과 렌더링 시점 모두 거친다.
lib/usecase/enter-with-api-key.ts	FN01 · FN03 · FN04 · FN05 · FN06 · FN11 · FN13	Key 기록 · 조회의 단일 진입점. 화면 계층은 localStorage 를 참조하지 않는다.
lib/usecase/plan-recipe.ts	FN03 · FN04 · FN05 (FN06 · FN07 · FN12 는 읽기 만)	레시피 생성 4개 경로가 모두 finish() 로 수렴한다.
lib/adapters/gemini-recipe-gateway.ts	FN03 · FN04 · FN05	음성 · 영상 · 재료 입력이 같은 call() 을 사용한다. responseSchema 만 다르다.
lib/adapters/browser-recipe-draft-store.ts	FN03 · FN04 · FN05 · FN12 (FN06 · FN07 은 읽기)	화면 간 전달되는 레시피 1건이 항상 이 위치를 경유한다.
lib/adapters/supabase-server-client.ts	FN02 · FN07 · FN08 · FN09 · FN10 · FN11 · FN13	서버용 Supabase 클라이언트. 쿠키 첨부을 이 위치에서만 처리한다.
lib/adapters/supabase-post-reader.ts	FN08 · FN09 · FN13	조회 전용. mine · canRemove 등 권한 판정값을 서버가 계산해 전달한다.
app/actions/post.ts	FN07 · FN08 · FN09 · FN10	게시글 · 댓글 · 좋아요 · 프로필 서버 액션을 한 파일에 모았다.
components/community/community-chrome.tsx	FN07 · FN08 · FN10 · FN11 · FN12 · FN13	커뮤니티 계열 7개 화면의 헤더 · 푸터.
components/post/post-actions.tsx	FN07 · FN09	좋아요 · 즐겨찾기 버튼과 수정 · 삭제 버튼을 함께 렌더링한다.

기능 목록 외 파일

기능별 흐름도 13개 항목이 다루지 않는 파일이다. 누락이 아니라 **실행 흐름이 없거나**, 소개 화면 · UI 구성 요소처럼 특정 기능에 종속되지 않는 파일이다.

파일	설명
lib/domain/measure.ts · components/pick/measure-tool.tsx	계량 단위 변환 도구. /pick 의 pick-rail 내부.
lib/recipe-card.ts	표지 이미지를 Canvas 로 렌더링. FN07 의 write-form 이 호출한다.
lib/site-content.ts · cook-content · post-content · post-copy · ask-content · write-content · shelf-content	화면 문구만 담긴 데이터 파일. 실행 흐름 없음.
components/hero · features · pricing · faq · closing · showcase · site-header · brand	소개 화면(/) 구성 요소.
components/icons.tsx	아이콘 컴포넌트. 대부분의 화면이 사용한다.
components/pick/pick-rail · start/start-head · shop/recipe-steps · post/post-recipe · post/cook-this · community/community-hero · community/ask-link	화면 구성 요소. 외부 통신이 없거나 상위 기능의 일부를 렌더링한다.
app/layout.tsx	전체 화면 레이아웃. 폰트와 메타데이터를 설정한다.
app/community2/page.tsx	디자인 시안 비교용 화면. 서비스 흐름 외부.

소스 132개 파일 가운데 위 표에 없는 파일은 모두 1장 계층 지도에 포함되어 있다. lib/adapters/pending-auth-gateway.ts 는 서버 미연결 시 대체용 게이트웨이이므로 FN02 칸에 함께 표기했다.

D1 Browser Storage (localStorage) — 서버로 전송하지 않음

ID	키	저장 값	기록 위치	조회 위치	사용 기능 · 비고
D1-1	cookpilot.gemini-key	Gemini API Key	lib/usecase/enter-with-api-key.ts:F4 ▶ lib/adapter/browser-api-key-store.ts:F4	lib/usecase/enter-with-api-key.ts:F7 (findSavedKey)	FN01 기록 · FN03-04-05-06-11-13 조회
D1-2	cookpilot.cook-setup	음성 성별 · 말투 · 응답 속도	lib/usecase/choose-cook-setup.ts:F1 ▶ lib/adapter/browser-cook-setup-store.ts:F3	lib/usecase/choose-cook-setup.ts:F4 (findCookSetup)	기능 목록 외 공통 단계 기록 · FN06-FN13 조회
D1-3	cookpilot.recipe-draft	진행 중인 레시피 1건	lib/usecase/plan-recipe.ts:F16 ▶ lib/adapter/browser-recipe-draft-store.ts:F3	lib/usecase/plan-recipe.ts:F18 (findDraft)	FN03-04-05-12 기록 · FN06-FN07 조회 — 화면 간 전달 값
D1-4	cookpilot.recipe-shelf	저장한 레시피 항목 목록	lib/usecase/keep-recipe-shelf.ts:F10 · F15 · F18	lib/usecase/keep-recipe-shelf.ts:F1 · F2	FN12
D1-5	cookpilot.comments	예시 게시글의 브라우저 저장 댓글	lib/usecase/discuss-post.ts:F1 (sayOnPost)	lib/usecase/discuss-post.ts:F9 (findMyComments)	FN08 — 테이블에 없는 예시 게시글 16건 전용
D1-6	cookpilot.chat-id	진행 중 대화 ID	lib/usecase/ask-kitchen.ts:F13 (keepChatId)	lib/usecase/ask-kitchen.ts:F12 (findChatId)	FN11

D2 Supabase 테이블

ID	테이블 이름	저장 값	기록 위치	조회 위치	사용 기능 · 비고
D2-1	profiles	display_name · avatar_url	lib/adapter/supabase-profile-gateway.ts:F1 (rename · setAvatar)	lib/adapter/supabase-post-reader.ts:F7 (조인) · app/community/page.tsx	FN10 기록 · FN07-FN08-FN13 조회
D2-2	posts	제목 · 요약 · 본문 · 카테고리(태그) · 표지 URL · like_count · published	lib/adapter/supabase-post-gateway.ts:F3 (insert) · F5 (rpc set_post_published) · F8 (update) · F11 (delete)	lib/adapter/supabase-post-reader.ts:F6 (livePosts) · F11 (livePost) · F19 (postAuthorId)	FN07 기록 · FN08-FN09-FN13 조회
D2-3	post_likes	(post_id, user_id)	lib/adapter/supabase-reaction-gateway.ts:F4 (insert / delete)	lib/adapter/supabase-reaction-gateway.ts:F8 (myReactions)	FN09. 트리거 sync_post_like_count 가 posts.like_count 를 ±1
D2-4	post_bookmarks	(post_id, user_id) — 본인만 조회	lib/adapter/supabase-reaction-gateway.ts:F4	lib/adapter/supabase-reaction-gateway.ts:F8	FN09. 집계하지 않으며 타 사용자에게 노출되지 않는다
D2-5	post_comments	게시글 ID · 작성자 · 본문 · 수정 시각	lib/adapter/supabase-comment-gateway.ts:F5 (insert) · F8 (update) · F12 (delete)	lib/adapter/supabase-post-reader.ts:F14 (postComments)	FN08. mine · canRemove · edited 는 조회 계층이 계산해 전달
D2-6	chats	대화 세션	lib/adapter/supabase-chat-log.ts:F5 (open)	확인 필요 — 목록 조회 코드가 없다	FN11
D2-7	messages	대화 메시지 (role · content)	lib/adapter/supabase-chat-log.ts:F6 (add)	lib/adapter/supabase-chat-log.ts:F7 ▶ rpc('chat_messages') ← app/api/chat/[id]/route.ts:F3	FN11. 테이블을 직접 조회하지 않고 RPC 함수를 거친다

D3 Supabase Storage 버킷

ID	버킷 이름	저장 값	기록 위치	조회 위치	사용 기능 · 비고
D3-1	post-covers	레시피 표지 png (<uuid>/<시각>.png)	lib/adapter/browser-cover-upload.ts:F6 ▶ 브라우저 직접 업로드	posts.image_url 을 통해 렌더링	FN07 — 서버 우회 ②
D3-2	avatars	프로필 이미지 webp 256px (<uuid>/<시각>.webp)	lib/adapter/browser-avatar-upload.ts:F9 ▶ 브라우저 직접 업로드	profiles.avatar_url 을 통해 렌더링	FN10 — 서버 우회 ②

D4 바깥 API

ID	외부 시스템	호출 주체	호출 위치	사용 기능 · 비고
D4-1	generativelanguage.googleapis.com — REST generateContent	브라우저 ▶ 직접	lib/adapter/gemini-recipe-gateway.ts:F6	FN03 · FN04 · FN05. Key 를 x-goog-api-key 헤더로 전달
D4-2	generativelanguage.googleapis.com — WebSocket BidiGenerateContent	브라우저 ▶ 직접	lib/adapter/gemini-live-gateway.ts:F3 (세션) · F20 (미리듣기)	FN06. 음성 모델은 voiceFor[gender][tone] 에서 결정
D4-3	generativelanguage.googleapis.com — 임베딩 · 응답 생성	서버 ▶	lib/adapter/langchain-rag.ts:F29 (chain.invoke)	FN11. 브라우저가 전달한 Key 를 서버가 사용
D4-4	Pinecone 벡터 색인	서버 ▶	lib/adapter/langchain-rag.ts:F28 (retriever.invoke) · F13 (addDocuments)	FN11. Pinecone Key 가 서버에만 있어 이 경로만 서버를 경유
D4-5	i.ytimg.com 썸네일	브라우저 ▶ 직접 (조회 전용)	lib/domain/recipe.ts:F30 (youtubeThumb) → components/shop/youtube-source.tsx	FN04. URL 생성만 하며 요청은 img 태그가 수행
D4-6	쇼핑몰 검색 (naver · coupang · kurly · ssg)	브라우저 ▶ 새 창	lib/domain/shopping.ts:F2 (searchUrl) · F4	FN03 계열의 /shop. URL 생성 후 사용자가 새 창으로 연다
D4-7	브라우저 speechSynthesis (내장 · 외부 서버 아님)	브라우저 안	lib/adapter/browser-speech.ts:F13	FN06 의 3순위 대비 경로

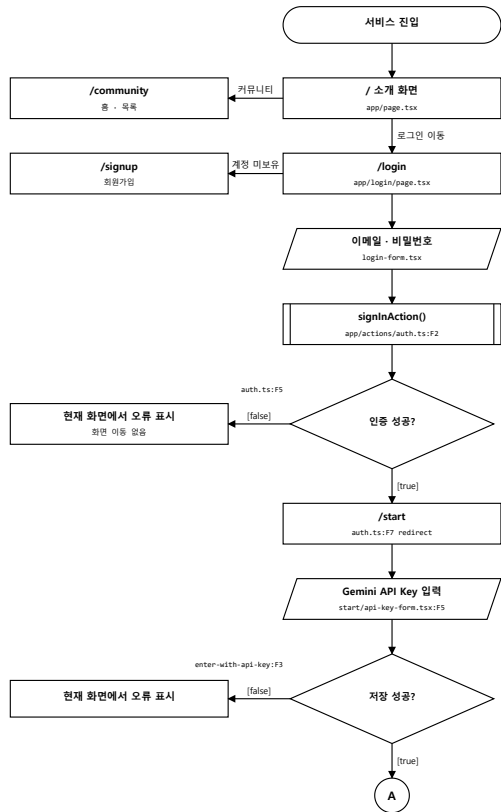
▶ 서버를 거치지 않는 길 (따로 표시)

▶ 브라우저 → 외부 (서버 미경유)	코드 위치	서버 미경유 사유
① 브라우저 ▶ Gemini REST	lib/adapter/gemini-recipe-gateway.ts:F6	Gemini Key 가 브라우저에만 저장된다. 서버가 접근할 수 없다.
① 브라우저 ▶ Gemini WebSocket	lib/adapter/gemini-live-gateway.ts:F3 · F20	실시간 오디오이므로 서버를 경유하면 지연이 발생한다.
② 브라우저 ▶ Supabase Storage (post-covers)	lib/adapter/browser-cover-upload.ts:F6	업로드는 장시간 소요된다. 서버를 점유하면 화면이 응답하지 않는다.
② 브라우저 ▶ Supabase Storage (avatars)	lib/adapter/browser-avatar-upload.ts:F9	동일한 이유. 서버에는 업로드 완료 후 URL 만 전달한다.
③ 브라우저 ▶ localStorage (D1 전부)	lib/adapter/browser-*-store.ts	서버에 저장할 값이 아니다. 계정 없이도 동작해야 한다.

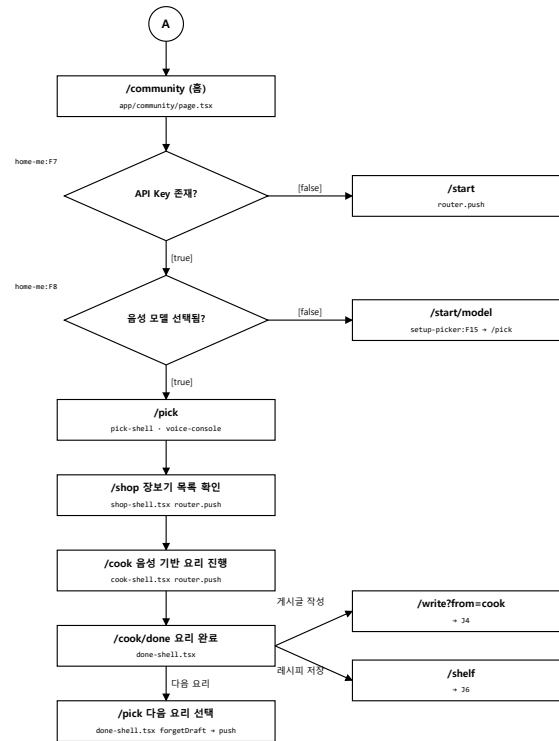
예외 — **FN11 만** 브라우저가 **Gemini Key** 를 **서버로 전달한다**. Pinecone Key 가 서버에만 있어 검색을 서버가 수행하고, 검색 결과로 응답을 생성하는 작업까지 서버에서 처리하기 때문이다. (components/ask/ask-shell.tsx:F8 ▶ app/api/chat/route.ts:F4)

J1 진입 · J2 주요 요리 흐름 · J7 접근 권한 검사

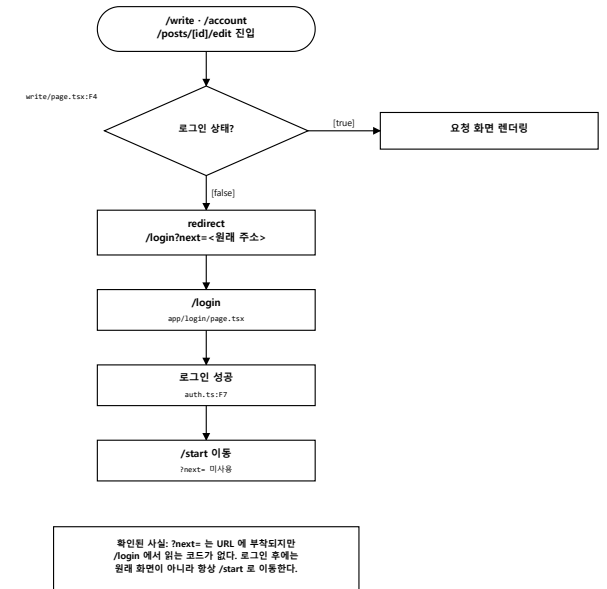
J1 진입 — 소개 화면에서 API Key 입력까지



J2 주요 요리 흐름 — 홈에서 요리 완료까지

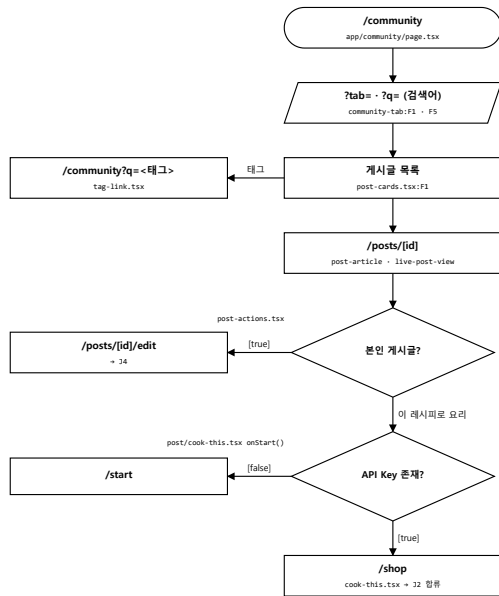


J7 접근 권한 검사 (세 화면 공통)

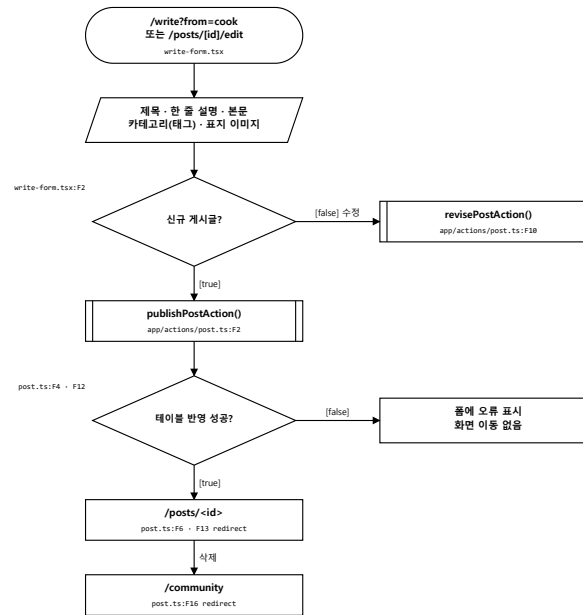


J3 커뮤니티 · J4 게시물 · J5 AI 요리 상담 · J6 저장 레시피

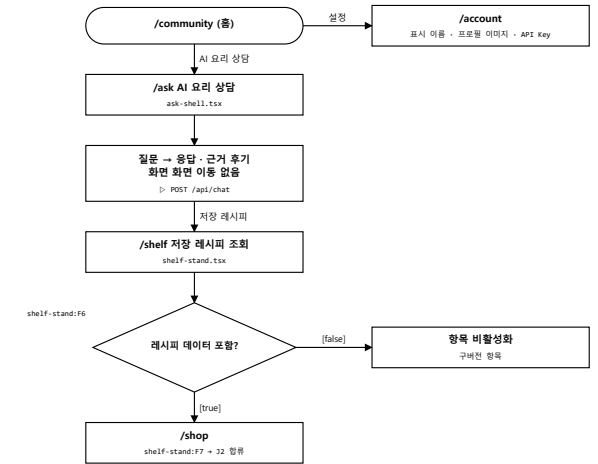
J3 커뮤니티 · 게시물 조회



J4 게시물 작성 · 수정 · 삭제

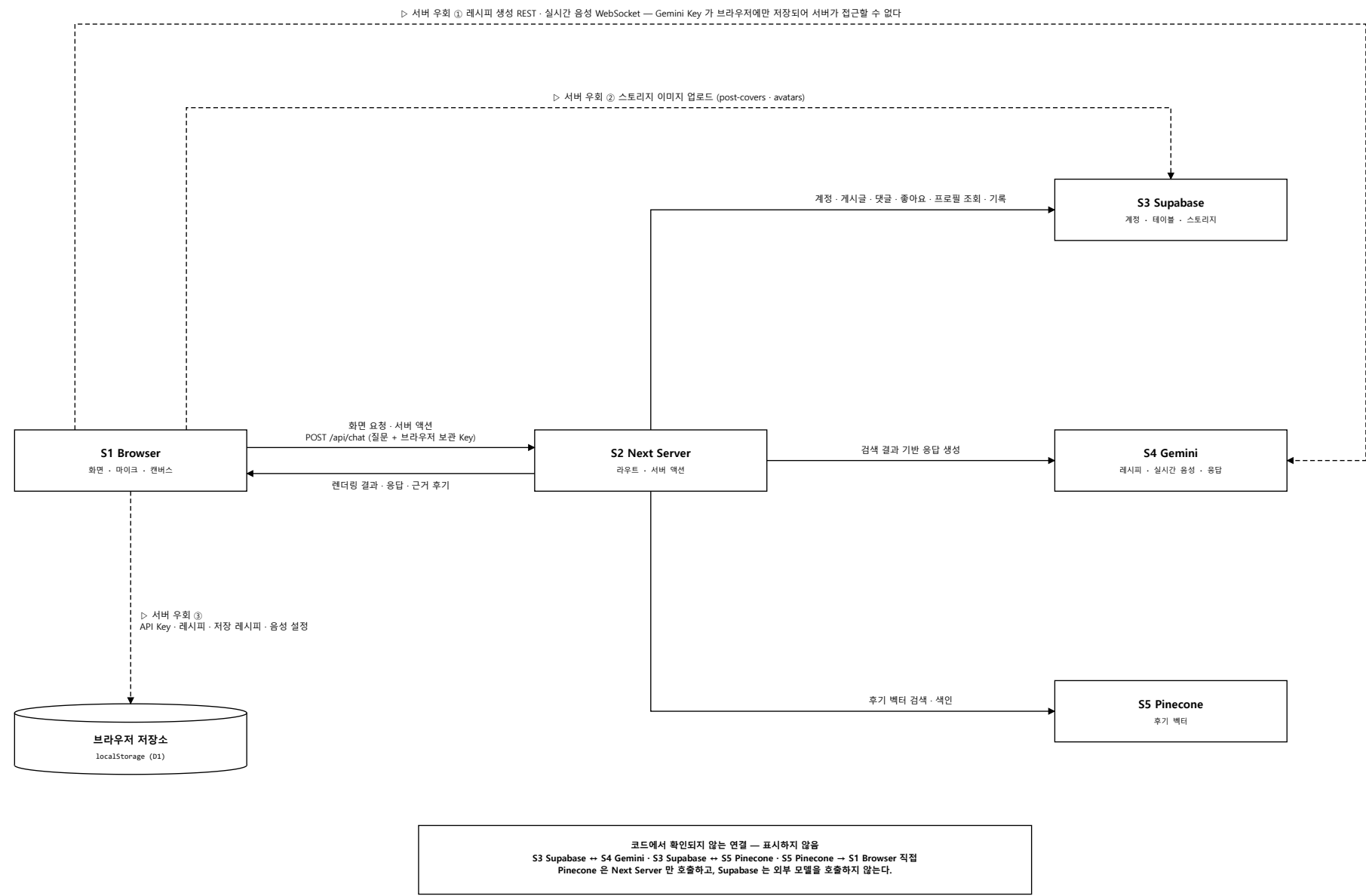


J5 AI 요리 상담 · J6 저장 레시피 · J7 계정



뒤로 가기 · 재진입은 코드에서 확인된 것만 그렸다. done-shell 이 forgetDraft() 로 임시 레시피를 삭제하고 /pick 으로 돌아가는 경로, write-form 의 '취소' 가 /posts/{id} 또는 /community 로 돌아가는 경로다. 브라우저 뒤로 가기를 처리하는 코드는 없다 — **확인 필요**.

S1 Browser · S2 Next Server · S3 Supabase · S4 Gemini · S5 Pinecone



계층 · 구현 위치 · 함수명 · 입력 · Process · 출력

분량 초과 — 이 표는 한두 화면에 들어가지 않는다. 축약하지 않고 기능별 계층 진입 함수만 수록했다(77개). 하위 함수는 docs/flow/flow.md 의 같은 FN 항목과 코드의 [F#] 주석을 참조한다. Process 는 코드에 실제로 구현된 처리만 기재했다.

계층	구현 위치	함수명	입력	Process	출력
FN01 Gemini API Key 입력					
L6	components/start/api-key-form.tsx :: onSubmit() :F5	onSubmit()	폼 입력 문자열	enterWithApiKey() 호출 후 결과로 화면 분기	저장 완료 카드 또는 오류 메시지
L2	lib/usecase/enter-with-api-key.ts :: enterWithApiKey() :F1	enterWithApiKey()	raw, store	checkApiKey → [false] 그대로 반환 / [true] ▷ store.save	{ok:true,key} 또는 {ok:false,problem reason}
L1	lib/domain/api-key.ts :: checkApiKey() :F1	checkApiKey()	raw: string	trim → 공백 여부 판정	{ok:true,key} 또는 {ok:false,problem:'empty'}
L3	lib/adapters/browser-api-key-store.ts :: save() :F4	save()	key	▷ localStorage.setItem('cookpilot.gemini-key')	boolean
L2	lib/usecase/enter-with-api-key.ts :: findSavedKey() :F7	findSavedKey()	store	▷ store.load() — 읽기 실패 시 null 과 동일하게 처리	ApiKey 또는 null
FN02 로그인 · 회원가입					
L4	app/actions/auth.ts :: signInAction() :F2	signInAction()	prev, formData	field() 로 값 추출 → signIn() → [true] revalidatePath 후 redirect	AuthFormState 또는 redirect('/start')
L2	lib/usecase/sign-in.ts :: signIn() :F1	signIn()	rawEmail, rawPassword, gateway	checkCredentials → [false] 반환 / [true] ▷ gateway.signIn()	{ok:true,email} 또는 오류 코드
L1	lib/domain/credentials.ts :: checkCredentials() :F1	checkCredentials()	rawEmail, rawPassword	빈 값 · @ 포함 여부 · 빈 비밀번호를 순서대로 검사	{ok:true,email,password} 또는 problem
L1	lib/domain/credentials.ts :: checkNewCredentials() :F8	checkNewCredentials()	rawEmail, rawPassword, rawConfirm	위 검증 + 8자 미만 · 확인 필드 불일치	{ok:true,...} 또는 problem
L1	lib/domain/display-name.ts :: checkDisplayName() :F1	checkDisplayName()	raw: string	표시 이름 길이 · 형식 판정	{ok:true,name} 또는 problem
L3	lib/adapters/supabase-auth-gateway.ts :: signIn() :F4	signIn()	email, password	▷ supabase.auth.signInWithPassword() → 오류를 코드로 분류	{ok:true} 또는 {ok:false,reason}
FN03 음성 입력 레시피 생성					
L6	components/pick/voice-console.tsx :: start() :F6	start()	없음 (마이크 버튼)	openMic → PCM 조각 누적, 재클릭 시 wavBase64 로 변환	clip
L3	lib/adapters/browser-audio.ts :: openMic() :F2	openMic()	없음	▷ getUserMedia → 워크릿에서 Float32 를 pcm16 으로 변환 [반복]	MicResult (핸들) 또는 오류 코드

계층	구현 위치	함수명	입력	Process	출력
L2	lib/usecase/plan-recipe.ts :: planFromSpeech() :F1	planFromSpeech()	clip, servings, gateway, store	음성 길이 하한 미달 시 호출 생략 → > gateway.fromSpeech → finish()	{ok:true,recipe} 또는 오류 코드
L1	lib/domain/recipe.ts :: readRecipe() :F3	readRecipe()	value: unknown, fallbackServings	[반복] 재료 · 단계 중 유효 항목만 수집, readSource 로 출처 판정	{ok:true,recipe} 또는 problem
L1	lib/domain/recipe.ts :: readSource() :F15	readSource()	value: unknown	youtube(url 필수) / shelf / community(chef 필수) / 그 외	RecipeSource
L3	lib/adapters/gemini-recipe-gateway.ts :: call() :F5	call()	parts, system, schema	> POST generateContent (Key 는 헤더) → 200 이 아니면 오류 코드로 분류	parsed JSON 또는 오류 코드
L1	lib/domain/shopping.ts :: searchUrl() :F2	searchUrl()	mall, keyword	쇼핑몰별 검색 URL 조립	URL 문자열
FN04 YouTube 링크 레시피 생성					
L2	lib/usecase/plan-recipe.ts :: planFromYoutube() :F8	planFromYoutube()	raw, servings, gateway, store	readYoutubeUrl 로 선검증 — 실패 시 모델을 호출하지 않는다	{ok:true,recipe} 또는 오류 코드
L1	lib/domain/recipe.ts :: readYoutubeUrl() :F22	readYoutubeUrl()	raw: string	URL 파싱 후 host 가 youtube.com ·youtu.be 인지 검사	{ok:true,url} 또는 'empty' 'not-youtube'
L1	lib/domain/recipe.ts :: youtubeThumb() :F30	youtubeThumb()	url	youtubeVideoid(:F28) 로 11자 ID 추출 후 URL 생성	i.ytimg.com/... 또는 null
FN05 냉장고 재료 레시피 생성					
L2	lib/usecase/plan-recipe.ts :: findFridgeIdeas() :F12	findFridgeIdeas()	raw, servings, gateway	readFridgeItems → > gateway.fridgeIdeas — 저장하지 않는다	{ok:true,ideas} 또는 오류 코드
L2	lib/usecase/plan-recipe.ts :: planFromFridgeIdea() :F13	planFromFridgeIdea()	dish, raw, servings, gateway, store	보유 재료를 재파싱해 요리명과 함께 전달 → finish()	{ok:true,recipe} 또는 오류 코드
L1	lib/domain/recipe.ts :: readFridgeItems() :F32	readFridgeItems()	raw: string	분리 · 정규화 후 최대 20개 — 2개 미만이면 거절	{ok:true,items} 또는 'empty' 'too-few'
L1	lib/domain/recipe.ts :: readFridgeIdeas() :F37	readFridgeIdeas()	value: unknown	[반복] 항목별 title 검증, 무효 항목 제외, 5개 도달 시 종료	FridgeIdea[]
FN06 음성 기반 요리 진행					
L2	lib/usecase/cook-along.ts :: cookingBrief() :F1	cookingBrief()	brief (레시피 · 조리 단계)	세션 시작 시 1회 전달하는 시스템 지시문 생성	문자열
L2	lib/usecase/cook-along.ts :: spokenStep() :F5	spokenStep()	text	낭독용 문자열만 추출	문자열
L2	lib/usecase/cook-along.ts :: readStepNote() :F6	readStepNote()	index, total, text	해당 조리 단계 낭독 지시문	문자열
L3	lib/adapters/gemini-live-gateway.ts :: geminiLiveGateway() :F1	geminiLiveGateway()	apiKey	> WebSocket → setup(voiceFor[gender][tone]) → [반복] 오디오 조각 송수신	LiveVoiceGateway (open · send · say · close)
L3	lib/adapters/gemini-live-gateway.ts :: previewVoice() :F19	previewVoice()	apiKey, gender, tone, line, onAudio, onDone	> WebSocket 임시 연결 후 한 문장 낭독하고 종료	해제 함수
L3	lib/adapters/browser-speech.ts :: speakWithBrowser() :F8	speakWithBrowser()	text, gender, tone, onDone	pickVoice(:F2) 로 한국어 음성 선택 후 rate · pitch 적용	없음 (> speechSynthesis.speak)
L1	lib/domain/cook-progress.ts :: readStepCommand() :F20	readStepCommand()	text	의문형이면 null → [반복] 접사 제거 후 8자 이내 토큰만 대조	'next' 'prev' 'repeat' 또는 null

계층	구현 위치	함수명	입력	Process	출력
L1	lib/domain/cook-progress.ts :: readTimerRequest() :F10	readTimerRequest()	text	타이머 키워드가 있을 때만 시 · 분 추출 후 1~1440 범위 검사	분(number) 또는 null
L1	lib/domain/cook-progress.ts :: moveStep() :F1	moveStep()	now, by, total	단계 번호를 범위 내로 보정	number
L2	lib/usecase/choose-cook-setup.ts :: keepCookSetup() :F1	keepCookSetup()	setup, store	▷ store.save — 저장 성공 여부 반환	boolean
FN07 게시물 작성 · 수정 · 삭제					
L4	app/actions/post.ts :: publishPostAction() :F2	publishPostAction()	prev, formData	writePost → [true] revalidatePath('/community') 후 redirect	WriteFormState 또는 redirect('/posts/<id>')
L2	lib/usecase/write-post.ts :: writePost() :F1	writePost()	raw, gateway	readPostDraft → [false] 오류 코드 / [true] ▷ gateway.publish	{ok:true,id} 또는 오류 코드
L1	lib/domain/post-draft.ts :: readPostDraft() :F3	readPostDraft()	제목 · 요약 · 본문 · 카테고리 · 소요 시간 · 이미지 URL	길이 검증 6종 + 자체 버킷이 아닌 이미지 URL 은 null 로 대체	{ok:true,draft} 또는 problem
L1	lib/domain/recipe-to-post.ts :: draftFromRecipe() :F7	draftFromRecipe()	recipe, tone	bodyFrom [반복] · guessBadge [반복] · totalMinutes [반복]	PostDraft (품 초기값)
L3	lib/adapters/supabase-post-gateway.ts :: publish() :F3	publish()	draft	▷ posts insert → ▷ rpc('set_post_published') — 게시 전환 실패를 무시하지 않는다	{ok:true,id} 또는 'rejected'
L3	lib/adapters/browser-cover-upload.ts :: uploadCover() :F3	uploadCover()	png: Blob	▷ auth.getUser → '<uuid>/<시각>.png' ▷ storage.upload ▷ getPublicUrl	{ok:true,url} 또는 오류 코드
L2	lib/usecase/write-post.ts :: revisePost() :F6	revisePost()	id, raw, gateway	작성과 동일 검증 → ▷ gateway.revise — 영향 행 0이면 타 사용자 게시물	{ok:true} 또는 'rejected'
L2	lib/usecase/write-post.ts :: removePost() :F11	removePost()	id, gateway	▷ posts delete — 댓글 · 좋아요는 on delete cascade 로 함께 삭제	{ok:true} 또는 오류 코드
FN08 댓글					
L4	app/actions/post.ts :: commentAction() :F17	commentAction()	prev, formData	writeComment → at 증가로 입력칸 초기화 후 revalidatePath	TalkFormState
L2	lib/usecase/discuss-post.ts :: writeComment() :F11	writeComment()	postId, raw, gateway	checkComment → [false] 오류 코드 / [true] ▷ gateway.add	{ok:true} 또는 오류 코드
L1	lib/domain/post.ts :: checkComment() :F2	checkComment()	raw: string	공백 · 2자 미만 · 500자 초과	{ok:true,text} 또는 problem
L3	lib/adapters/supabase-post-reader.ts :: postComments() :F14	postComments()	postId, postAuthorId	▷ select(작성 시각 오름차순) → [반복] mine · canRemove · edited · when 계산	LiveComment[]
L2	lib/usecase/discuss-post.ts :: reviseComment() :F15	reviseComment()	commentId, raw, gateway	작성과 동일 검증 → ▷ update — 영향 행 0이면 타 사용자 댓글	{ok:true} 또는 'rejected'
L2	lib/usecase/discuss-post.ts :: dropComment() :F19	dropComment()	commentId, gateway	▷ delete — 삭제 권한은 RLS 정책이 판정	{ok:true} 또는 오류 코드
FN09 좋아요 · 즐겨찾기					

계층	구현 위치	함수명	입력	Process	출력
L4	app/actions/post.ts :: likeAction() :F25	likeAction()	postId, on	toggleLike → revalidatePath('/community') · '/posts/<id>'	{ok} — 실패 시 화면 상태 원복
L2	lib/usecase/react-to-post.ts :: toggleLike() :F2	toggleLike()	postId, on(화면이 계산한 다음 상태), gateway	looksLikePost(uuid) 로 예시 게시물 제외 후 > gateway.setLike	{ok:true,on} 또는 'rejected'
L3	lib/adaptersupabase-reaction-gateway.ts :: set() :F1	set()	테이블 이름, postId, on	> insert / delete — 23505(중복 키)는 성공으로 처리	{ok:true} 또는 오류 코드
L3	lib/adaptersupabase-reaction-gateway.ts :: myReactions() :F6	myReactions()	postId	> post_likes · post_bookmarks 를 Promise.all 로 병렬 조회	{liked, saved}
FN10 프로필 관리					
L4	app/actions/post.ts :: setPhotoAction() :F29	setPhotoAction()	prev, formData	changeMyPhoto → revalidatePath('/community') · '/account'	PhotoFormState
L2	lib/usecase/rename-me.ts :: changeMyPhoto() :F6	changeMyPhoto()	raw(이미지 URL), gateway	공백이면 이미지 제거 → readAvatarUrl → null 이면 'bad-url'	{ok:true} 또는 오류 코드
L1	lib/domain/avatar.ts :: readAvatarUrl() :F1	readAvatarUrl()	raw: unknown	https 로 시작하고 '/avatars/' 를 포함하는 URL 만 통과	url 또는 null
L3	lib/adaptersbrowser-avatar-upload.ts :: uploadAvatar() :F6	uploadAvatar()	file: File	> createImageBitmap → 중앙 정사각형 크롭 → 256px webp > storage.upload	{ok:true,url} 또는 오류 코드
L2	lib/usecase/rename-me.ts :: renameMe() :F1	renameMe()	raw, gateway	checkDisplayName(회원가입과 동일 규칙) → > gateway.rename	{ok:true,name} 또는 오류 코드
FN11 AI 요리 상담 (RAG)					
L5	app/api/chat/route.ts :: POST() :F4	POST()	request (질문 · 대화 이력 · 브라우저 보관 Key)	checkApiKey → langchainRag(Key) → askKitchen	JSON {turn, chatId} 또는 오류 코드
L2	lib/usecase/ask-kitchen.ts :: askKitchen() :F1	askKitchen()	raw, history, chain, log, chatId	checkQuestion → 대화 생성 → > log.add(질문) → chain.run → > log.add(응답)	{ok:true,turn,chatId} 또는 오류 코드
L1	lib/domain/ask.ts :: checkQuestion() :F1	checkQuestion()	raw: string	공백 · 2자 미만 · 500자 초과	{ok:true,text} 또는 problem
L1	lib/domain/ask.ts :: recentTurns() :F5	recentTurns()	history	최근 6턴만 유지 — 전량 전송 시 프롬프트 한도를 초과한다	ChatTurn[]
L1	lib/domain/ask.ts :: foldSources() :F7	foldSources()	refs	[반복] 동일 dishId 중복 제거	SourceRef[]
L3	lib/adapterslangchain-rag.ts :: run() :F22	run()	question, 최근 대화 턴	> retriever.invoke(Pinecone k=5) → > chain.invoke(Gemini) → refOf	{ok:true,text,sources} 또는 'key' 'no-hits'
L3	lib/adapterslangchain-rag.ts :: indexReviews() :F10	indexReviews()	reviews	[반복] reviewText 로 Document 생성 → [반복] 80건 단위로 > addDocuments	{ok:true,count}
L1	lib/domain/review.ts :: reviewText() :F9	reviewText()	review	후기 1건을 검색 대상 문자열로 변환 — 이 문자열이 벡터가 된다	문자열
L3	lib/adapterscsv-reviews.ts :: loadReviews() :F4	loadReviews()	없음	> samples/reviews.csv 읽기 → [반복] readReviewRow, 무효 행은 건너뛴	{reviews, skipped}

계층	구현 위치	함수명	입력	Process	출력
FN12 저장 레시피 관리					
L2	lib/usecase/keep-recipe-shelf.ts :: shelveRecipe() :F7	shelveRecipe()	book, store	[반복] 동일 제목 제외 후 선두에 추가 → ▷ store.replace	{ok:true,count}
L2	lib/usecase/keep-recipe-shelf.ts :: unshelveBook() :F12	unshelveBook()	id, store	[반복] 해당 ID 제외 후 ▷ store.replace	{ok:true,count}
L2	lib/usecase/keep-recipe-shelf.ts :: restoreShelfBackup() :F16	restoreShelfBackup()	text, store	readBackup 실패 시 저장 항목을 변경하지 않는다 → ▷ store.replace	{ok:true,count} 또는 오류 코드
L1	lib/domain/recipe-shelf.ts :: readBackup() :F5	readBackup()	text	JSON 형식 · 버전 · [반복] 항목 구조 불일치 시 전체 거절	{ok:true,books} 또는 'broken' 'version' 'shape'
L1	lib/domain/recipe-shelf.ts :: writeBackup() :F2	writeBackup()	books	버전을 포함해 JSON 문자열로 직렬화	문자열
FN13 커뮤니티 목록 · 검색 · 태그					
L5	app/community/page.tsx :: CommunityPage() :F1	CommunityPage()	searchParams(tab · q)	resolveTab · resolveQuery → livePosts → matchesQuery 로 필터링	화면(JSX)
L1	lib/domain/community-tab.ts :: resolveTab() :F1	resolveTab()	raw: string string[] undefined	배열이면 마지막 값 → 미등록 값이면 DEFAULT_TAB	CommunityTab
L1	lib/domain/community-tab.ts :: matchesQuery() :F8	matchesQuery()	fields(제목 · 작성자 · 태그 · 요약), query	squash(공백 제거 + 소문자) 후 [반복] 한 필드라도 일치하면 통과	boolean
L3	lib/adaptersupabase-post-reader.ts :: livePosts() :F6	livePosts()	limit	▷ select + profiles 조인 → 컬럼 미존재 오류 시 해당 컬럼 제외 후 재조회	LivePost[] (실패하면 빈 배열)
L1	lib/domain/post.ts :: photoPath() :F1	photoPath()	postId	예시 게시물 이미지 경로를 규칙으로 생성	경로 문자열